

Neural-First CDCL Portfolios with Online-Consistent Event Guidance

Anonymous Authors

Abstract

Learning-guided SAT solvers face a practical tension. One-shot neural guidance is efficient because the neural model is queried only once before search, but it remains open-loop: it cannot react to what the CDCL solver observes during search. Fully dynamic neural branching can condition on solver state, but frequent neural calls inside the branching loop introduce substantial wall-clock overhead and can harm complete-solver reliability. This paper studies a conservative neural-first portfolio: a low-budget neural-guided stage uses CDCL event traces to decide whether feedback should be activated, and unsolved instances fall back to a strong CDCL solver.

We propose an Online-Consistent Selector for event-conditioned SAT guidance. The neural first stage obtains a one-shot guidance prior, runs short warmup probes to collect CDCL events, and applies a conservative risk/recovery controller to choose between the one-shot path and an event-adapted guidance path. A guarded Local Boundary Correction reopens a small set of over-conservative selector decisions under a candidate manifest. We then use this guarded neural stage in a simple sequential portfolio: run Local Boundary Correction for at most 5s, then run CaDiCaL for at most 55s on unsolved instances.

On 200 held-out 3SAT-400 instances, the end-to-end Local5–CaDiCaL55 portfolio solves 79–80/200 over three full repeats. However, repeated standalone CaDiCaL 60s runs solve 75, 80, and 80 instances, so the matched portfolio deltas are +4, 0, and 0. A repeated-CaDiCaL overlap audit finds three Local-solved instances that CaDiCaL misses in all repeats, two of which are already solved by the Online-Consistent Selector; the Local-only strict boundary contribution is `3sat_188.cnf`. A further baseline audit with the existing March solver changes the interpretation: under a strict 60s wall-clock count, March solves 184/200 in each of three repeats and solves all three CaDiCaL-strict complement instances. Neural-stage results therefore explain a risk-control mechanism inside the neural-guided Glucose workflow, not a performance advantage over strong SAT baselines.

1 Introduction

Modern CDCL SAT solvers are highly optimized, and even useful learned guidance can be harmful if it is inserted at the wrong point in the solving loop. One-shot neural guidance is attractive because it queries a graph neural network once before search and then lets the solver run normally, but this efficiency comes with an open-loop limitation: the guidance cannot react to conflicts, propagations, learnt clauses, or activity patterns observed during search. Dynamic neural branching can use such solver state, but frequent neural inference inside the branching loop creates wall-clock overhead and can introduce reliability risks for a complete solver. This paper asks whether neural guidance can be used as a cheap complementary first stage for a strong CDCL solver, rather than as an unconditional replacement for CDCL search.

The central difficulty is asymmetric risk. A learned adapter can recover hard instances or reduce long-tail runtime, but it can also lose instances that the base one-shot path solves or slow

easy instances enough to erase the gains. Therefore, a useful feedback mechanism should not simply maximize adapter usage. It should intervene only when observed solver evidence suggests meaningful recovery potential and low slowdown or lost-solution risk.

We study this problem in the setting of RLAF-style one-shot SAT guidance [9]. The neural stage first predicts a guidance prior from the CNF graph. It then performs low-budget warmup probes under that prior and collects CDCL event traces. These traces provide variable-level and graph-level evidence about the actual search trajectory. An Online-Consistent Selector uses this evidence to decide whether to run the final neural-stage solve with event-adapted guidance or with the safer one-shot guidance. Unsolved instances are then passed to CaDiCaL in a fixed second stage.

Our main contribution is not a more complex branching model. Instead, it is a conservative neural-first portfolio with an evidence-driven control layer:

1. We propose an event-conditioned online selector that uses real CDCL event traces to decide whether neural feedback should be activated in a low-budget first stage.
2. We introduce conservative risk control for neural SAT guidance, separating recovery potential from slowdown and lost-solution risk.
3. We audit the resulting neural-first/CDCL-second portfolio against repeated CaDiCaL and March baselines, exposing the boundary where neural-guided Glucose improvements stop being competitive with stronger SAT solvers.

2 Related Work

Learning-guided SAT solving. Neural SAT methods inject learned predictions into complete solvers in several ways. NeuroSAT showed that message passing can learn useful SAT representations from satisfiability supervision [8]. NeuroCore used neural unsat-core predictions to guide high-performance SAT solvers through variable activity [7]. Graph-Q-SAT studied reinforcement learning for GNN-based branching heuristics [5]. NeuroBack and RLAF emphasize practical low-frequency guidance by predicting backbones, phases, weights, or polarities before search rather than querying a neural model at every decision [1, 9]. Our work follows this practical line but differs in one key respect: the guidance decision is not purely static. We use the one-shot neural guidance prior to collect warmup CDCL event evidence, then decide conservatively whether event-conditioned feedback should be activated for the final solve.

Solver-state and trace-based learning. Several systems use solver-internal information rather than only static CNF structure. Graph-Q-SAT and RDC-SAT condition learned decisions on evolving solver states [5, 12], while trace-supervised systems such as ImitSAT and NeuroSelect show that CDCL traces, clauses, activities, and other solver artifacts can provide useful learning signals [3, 2]. Our use of solver feedback is narrower. We do not learn a full branching policy, splitting policy, or clause-deletion policy. Instead, we collect event traces at configured intervention points and use them for a single risk-controlled activation decision.

Risk-controlled neural intervention and portfolios. Portfolio and per-instance algorithm selection have long been important in SAT solving [11, 4]. SATzilla-style systems choose among solvers or configurations using instance features. Our setting is different: the first stage is a neural-guided CDCL workflow that decides whether event feedback should be activated, and the

second stage is a fixed CaDiCaL fallback. The decision is still risk-sensitive. A neural heuristic that improves a few hard instances can be undesirable if it loses solutions or slows many easy instances. We therefore separate recovery evidence from slowdown and lost-solution risk, and report solved count, wall-clock mean, per-instance overlap, and portfolio-only evidence rather than a single aggregate metric.

Event-conditioned identity. Static message-passing GNNs are limited by neighborhood aggregation, often analyzed through the Weisfeiler-Leman hierarchy [10, 6]. SAT instances can contain variables that are structurally similar in the CNF graph but behave differently once CDCL search begins. CDCL event traces provide search-induced distinctions through decisions, propagations, conflicts, learnt-literal occurrences, and activity. We do not claim that these traces solve GNN expressivity limits; rather, they provide practical online evidence for deciding when feedback is useful.

3 Method

3.1 Overview

We study how to use neural guidance inside a CDCL SAT solver without turning every branching decision into a neural inference call. The key design is to keep neural intervention sparse and evidence-driven. The solver first obtains a one-shot neural guidance prior, then runs short warmup probes to collect event traces from the actual CDCL trajectory. These traces are converted into event-conditioned solver states and graph-level selector features. A conservative Online-Consistent Selector decides whether the solver should continue with the neural adapter or fall back to the safer one-shot guidance path. Finally, a guarded Local Boundary Correction can reopen a small set of over-conservative boundary cases.

3.2 Problem Setup

Given a CNF formula F , the base neural solver encodes the formula as a bipartite literal-clause graph and predicts a variable-level guidance vector. In our implementation, this guidance is passed to a weighted CDCL solver as per-variable polarity and weight comments in the DIMACS input. The one-shot baseline is therefore a neural guidance baseline: it uses a GNN prediction once before solving, but does not use feedback refinement after observing a solver trajectory.

Let $g_\theta(F)$ denote the one-shot GNN prediction and let $S(F, g_\theta(F); B)$ denote running the CDCL solver with guidance $g_\theta(F)$ under budget B . A naive dynamic approach would repeatedly call the GNN throughout the solve. We avoid this design. Instead, we collect solver-side evidence at a small number of fixed warmup budgets, make one intervention decision, and then run one final guided solve.

3.3 Event-Conditioned Solver State

The warmup rollout records CDCL event statistics by enabling event collection in the solver. At the variable level, the solver exports decision occurrences, propagation occurrences, conflict-literal occurrences, learnt-clause literal occurrences, and variable activity. The event state maps these raw counts into stable per-variable features using log-scaled counts, rates, and ranks. This produces a variable-level event tensor attached to the graph before the event adapter is evaluated. In the

full400 paper configuration, we use the non-polarity enhanced event state; polarity-specific event features are diagnostic and are not part of the final method story.

For a variable v , let e_v denote the event-conditioned state extracted after warmup. The adapter receives the base variable embedding together with e_v and predicts a residual correction to the one-shot guidance. The event state can be updated with momentum across refinement rounds; the final full400 configuration uses one feedback round with state momentum 0.5.

3.4 Online-Consistent Selector

The selector addresses the central risk in neural feedback: the event adapter can help hard instances, but it can also slow down or lose instances that the one-shot path would already solve. Therefore, the selector operates at the instance level and decides whether the adapted branch should be used at all.

The final selector is online-consistent: its features are built from the same online warmup evidence available at inference time, rather than from stale or mismatched caches. For each warmup point, the system extracts graph-level statistics such as decisions, propagations, CPU time, base guidance distribution summaries, event entropy, top-mass concentration, and correlations between the one-shot guidance and event activity. With multiple warmup conflict points, it also extracts drift features between neighboring points.

The selector is a conservative two-stage controller. It computes a risk score p_{risk} , a recovery score p_{recovery} , and optionally a slowdown-veto score p_{slowdown} . The main decision rule is

$$\text{use_adapter} = [p_{\text{risk}} < \tau_{\text{risk}}] \wedge [p_{\text{recovery}} \geq \tau_{\text{recovery}}].$$

If a slowdown veto head is present, the selector also requires $p_{\text{slowdown}} < \tau_{\text{slowdown}}$. This form makes the claim precise: the selector is not a generic accuracy-maximizing classifier. It is a risk controller whose first responsibility is to avoid unsafe adapter activation, then recover hard or timeout cases when evidence is strong enough.

3.5 Training From Same-Evidence Branches

Training uses same-evidence counterfactual traces. The system first runs a warmup rollout under the same one-shot guidance and collects event evidence. It then evaluates two branches from this shared evidence: a base branch that continues with the one-shot guidance path and an adapter branch that applies event-conditioned neural correction. Because the current Glucose wrapper cannot serialize and restore the full internal CDCL trail and clause database, these are same-evidence branch counterfactuals rather than exact in-process CDCL clone continuations.

Labels separate recovered timeouts, hard speedups, lost solutions, easy slowdowns, general slowdowns, neutral cases, and warmup-solved cases. The risk head learns when adapter activation is unsafe, while the recovery head learns when adapter activation has meaningful upside. At inference time, only the selected branch is executed in the final solve.

3.6 Local Boundary Correction

The Online-Consistent Selector is intentionally conservative, which creates a specific failure mode: it can close adapter branches that earlier compact evidence indicates were useful. These cases are the `new_closed_old_on` boundary. Local Boundary Correction is a guarded repair for this narrow failure mode. It is not a new global model. It only triggers when an instance is present in a candidate manifest and satisfies a small rule over warmup features.

The official full400 boundary correction uses the 750-feature guarded rule:

$$\begin{aligned} \text{local_reopen_candidate} &\geq 1, \\ \text{warmup_c1000_minus_warmup_c750_decisions} &\geq 294, \\ \text{warmup_c2000_rho_event_corr} &\geq 0.0365. \end{aligned}$$

The candidate guard is essential. It prevents the local rule from becoming a global threshold sweep over the full test set.

Let $m(F)$ indicate membership in the candidate manifest and let $r(F)$ indicate the local reopen rule. The final adapter decision is

$$\text{use_adapter_final} = \text{use_adapter} \vee (\neg \text{use_adapter} \wedge m(F) \wedge r(F)).$$

We present this mechanism as a boundary correction ablation. On the current full400 robustness protocol it matches the Old Compact solved count and reduces mean time by reopening a few over-conservative boundary cases. A no750 variant is diagnostic only and is not the official full400 patch.

3.7 Inference Workflow

The boxed procedure below summarizes the inference-time workflow. The key property is that neural calls are low frequency: the method performs one base prediction, a small number of warmup probes to gather event evidence, one selector decision, and one final guided solve.

Procedure 1: Online-consistent event-conditioned guidance

1. Given formula F , run the base GNN once to obtain one-shot guidance $w_0 = g_\theta(F)$.
2. For each configured intervention point $b \in \mathcal{B}$, collect a cumulative warmup probe under w_0 up to conflict budget b .
3. Encode the resulting CDCL event trace as variable event state and graph-level selector features.
4. Compute p_{risk} and p_{recovery} from online warmup features.
5. Set u to true only if risk is below threshold and recovery is above threshold.
6. If the candidate manifest contains F and the local boundary rule is satisfied, set u to true as guarded boundary correction.
7. Run the final CDCL solve under event-adapted guidance when u is true; otherwise run under the one-shot guidance w_0 .

4 Experiments

We evaluate whether online-consistent conservative selection can provide a useful neural first stage for a strong CDCL portfolio under real wall-clock evaluation. The main questions are: how the end-to-end Local5–CaDiCaL55 schedule compares with standalone CaDiCaL 60s under repeated baselines, whether any apparent advantage is stable under repeated CaDiCaL runs, and how much of the portfolio-only evidence comes from neural-first complementarity rather than second-stage runtime-boundary behavior.

Table 1: End-to-end full400 portfolio and repeated CaDiCaL baseline on 200 held-out 3SAT-400 instances. The portfolio runs + Local Boundary Correction for at most 5s, then CaDiCaL for at most 55s on unsolved instances. CaDiCaL repeats are standalone 60s runs. The model, selector, correction guard, solver binaries, and time split are fixed.

Run	Portfolio	Local	C2 solves	CaDiCaL	Delta	C-only loss
0	79	40	39	75	+4	0
1	80	40	40	80	+0	0
2	80	40	40	80	+0	0

4.1 Experimental Setup

The main benchmark is `data/test/3sat/400/*.cnf`, containing 200 held-out 3SAT-400 instances. All full400 conclusions come from real wall-clock runs rather than offline estimates or stale selector caches. Standalone baselines use a nominal 60s solve limit; the external guard is 65s, so a small number of solver-reported CPU times may slightly exceed 60s. As solver-default references, unweighted Glucose solves 13/200 instances and the original standalone CaDiCaL run solves 75/200; repeated CaDiCaL 60s runs solve 75, 80, and 80 instances. We also audit the existing unweighted March binary for three full repeats. Because the current March runner uses only an external timeout, we report a strict 60s count by treating March solutions with wall time greater than 60s as timeouts.

The main portfolio runs guarded Local Boundary Correction for at most 5s, then runs CaDiCaL for at most 55s on instances not solved by the neural first stage. We also report neural-stage mechanism results comparing the one-shot neural guidance baseline, Old Compact, the Online-Consistent Selector, and guarded Local Boundary Correction. Pairwise Veto, compact-stable variants, polarity/SBE variants, and no750 boundary variants are treated as negative results or diagnostics, not as the main method.

4.2 Main Full400 Portfolio Results

Table 1 gives the main end-to-end portfolio result together with repeated standalone CaDiCaL 60s runs. The Local5–CaDiCaL55 portfolio solves 79–80/200 instances over three full400 repeats. The original CaDiCaL run solved 75/200, but repeated CaDiCaL runs solve 80/200; matched deltas are therefore +4, 0, and 0. The portfolio has zero CaDiCaL-only losses in all three repeats, but its solved-count advantage is boundary-sensitive rather than repeated-baseline robust.

Table 2 splits the portfolio-only evidence by source. This distinction is important: the paper does not claim that neural guidance alone contributes the entire original-run +4 delta, and the repeated-CaDiCaL audit shows that only one neural-first complement remains strict across all CaDiCaL repeats. The remaining portfolio-only evidence is second-stage CaDiCaL cutoff/runtime-boundary behavior or CaDiCaL repeat variance.

4.3 Instance-Level Behavior

The portfolio improvement is not uniform across all instances. Under the original CaDiCaL run, portfolio-only neural-first complement comes from `3sat_132.cnf`, `3sat_140.cnf`, and `3sat_25.cnf`. Repeated CaDiCaL 60s runs solve `3sat_132.cnf` and `3sat_25.cnf`, leaving `3sat_140.cnf` as the strict repeated-baseline complement within the original-run portfolio-only split. The runtime-

Portfolio advantage is boundary-sensitive on 3SAT-400

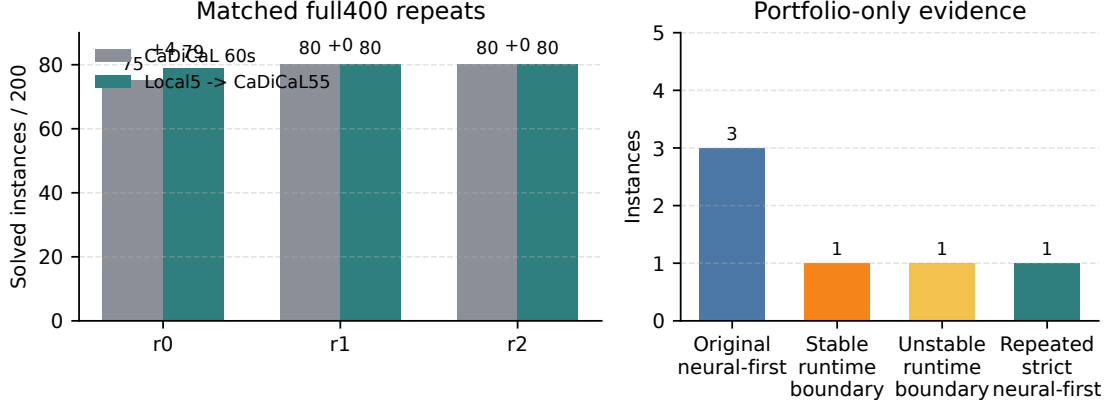


Figure 1: Portfolio and CaDiCaL repeat audit on 200 held-out 3SAT-400 instances. Left: the original CaDiCaL 60s run solves 75/200, but repeated CaDiCaL 60s runs solve 80/200, matching the best portfolio repeats. Right: original-run portfolio-only evidence split by source; within that portfolio-only split, repeated CaDiCaL reduces strict neural-first complement to one instance.

Table 2: Portfolio-only evidence under the original CaDiCaL 60s run, with repeated-CaDiCaL audit. Neural-first complement means + Local Boundary Correction solves the instance within the 5s first-stage budget while the original standalone CaDiCaL 60s run times out. Repeated CaDiCaL runs show that only one such instance remains unsolved by CaDiCaL across all repeats.

Evidence class	Inst.	PO reps	Repeated-CaDiCaL strict
Original-run neural-first complement	3	9	1
Stable second-stage runtime boundary	1	3	0
Unstable second-stage runtime boundary	1	2	0

boundary portfolio-only cases `3sat_111.cnf` and `3sat_48.cnf` are also solved by repeated standalone CaDiCaL 60s runs.

Table 3 gives the full repeated-CaDiCaL overlap audit. This audit gives a broader view because it is not restricted to instances that are portfolio-only under the original CaDiCaL run. Local Boundary Correction solves three instances that CaDiCaL misses in all three 60s repeats: `3sat_140.cnf`, `3sat_147.cnf`, and `3sat_188.cnf`. Online-Consistent Selector already solves `3sat_140.cnf` and `3sat_147.cnf`; the only strict Local-only gain beyond Online and repeated CaDiCaL is `3sat_188.cnf`. Conversely, the five one-shot timeouts recovered by Online are all solved by CaDiCaL in all three repeats, so they explain neural-workflow recovery but not strong-CDCL complementarity.

The March baseline further limits the claim. Under the strict 60s count, March solves 184/200 instances in each of three repeats with mean capped time 26.713s, and its strict solved/unsolved status is stable on all 200 instances. It solves `3sat_140.cnf`, `3sat_147.cnf`, and `3sat_188.cnf` in all three repeats, removing the CaDiCaL-strict complement set. Local Boundary Correction has no solved-count complement against repeated March strict-60. We therefore treat the neural results as evidence about risk-controlled feedback inside a neural-guided Glucose workflow, not as a strong-SAT-baseline performance result.

Table 3: Full repeated-CaDiCaL overlap audit on 200 held-out 3SAT-400 instances. Neural solved means solved in all three neural solver-seed runs. CaDiCaL unsolved means unsolved in all three standalone CaDiCaL 60s repeats. Local-only is the strict boundary-correction contribution beyond Online-Consistent Selector.

Overlap question	Count
CaDiCaL solved in any repeat / Local unsolved	29
CaDiCaL solved in all repeats / Local unsolved	26
Local solved / CaDiCaL unsolved in all repeats	3
Online solved / CaDiCaL unsolved in all repeats	2
Local-only over Online / CaDiCaL unsolved in all repeats	1
One-shot timeout recovered by Online; CaDiCaL solved all	5
Local Correction open set; CaDiCaL solved all	2/4

Table 4: Neural-stage full400 solver-seed robustness on 200 held-out 3SAT-400 instances. Values are means over Glucose seeds 1, 2, and 3. These rows explain the neural first stage used in the portfolio; they are not the top-line strong-CDCL result.

Method	Solved	Mean time (s)	Median time (s)	Solved std
One-shot	48.0	47.665	60.289	0.0
Online-Consistent Selector	53.0	46.324	60.366	0.0
Old Compact	54.0	46.278	60.373	0.0
+ Local Boundary Correction	54.0	45.914	60.369	0.0

Within the neural first stage, decision audits identify two important boundary classes. In `new_opened_old_off` cases, the new selector opens an adapter branch that the old compact selector closed, such as `3sat_140.cnf`, which drops from 41.083s to 4.339s. In `new_closed_old_on` cases, the new selector closes a branch that earlier evidence opened. This includes correctly avoided slowdowns such as `3sat_82.cnf`, but also over-conservative misses such as `3sat_46.cnf` and `3sat_196.cnf`. These boundary misses motivate the guarded Local Boundary Correction ablation.

4.4 Neural-Stage Stability Validation

The portfolio uses guarded Local Boundary Correction as its neural first stage. Table 4 reports the neural-stage full400 solver-seed robustness that explains this component. Online-Consistent Selector is stable over One-shot across seeds 1, 2, and 3, improving solved count from 48/200 to 53/200. Old Compact remains a strong matched neural-stage reference at 54/200. Local Boundary Correction matches Old Compact solved count and gives the lowest mean time, so its role is guarded boundary repair rather than an unrestricted neural main model.

Neural-stage stability validation uses only the final full400 per-instance results. We do not reuse old selector caches or retrain the main model. Table 5 reports runtime and solver-seed robustness for the neural stage. Same-seed repeats use three full400 reruns with the default seed. Solver-seed robustness uses Glucose seeds 1, 2, and 3.

The neural-stage stability analysis supports the same interpretation as the mechanism table. Online-Consistent Selector is stable over One-shot, Old Compact remains the strongest uncorrected neural-stage reference in solved count, and Local Boundary Correction matches Old Compact solved

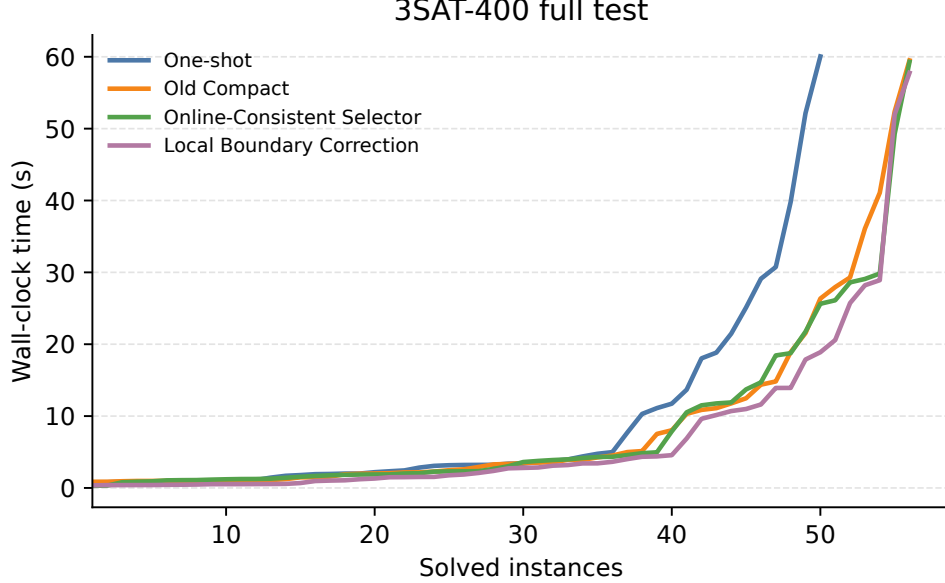


Figure 2: Neural-stage cactus plot over solved instances on the full 3SAT-400 test set. Curves sort solved instances by wall-clock solving time. Online-Consistent Selector recovers most of the One-shot timeout loss. Local Boundary Correction matches Old Compact solved count and further reduces mean time by repairing a small set of guarded boundary cases. This is a mechanism figure for the neural first stage, not the portfolio main result.

count while giving the lowest mean time. Portfolio-level stability is reported in Table 1.

4.5 Local Boundary Correction Ablation

Table 6 summarizes the boundary correction ablations used in the paper narrative. The official full400 patch uses `intervention_conflicts`: [500, 750, 1000, 2000] and the 750-feature decision rule. The no750 diagnostic opens a different neutral instance than the formal full400 patch and should not be mixed with the final full400 result.

4.6 Negative Results

The negative results help explain why the paper does not continue adding selector complexity. Pairwise Veto improves over One-shot but reaches only 52/200 solved instances, below the Online-Consistent Selector. Compact Stable is useful as a failed stability branch, but it does not replace Old Compact as the reference baseline. Polarity and SBE variants are unstable on 3SAT-400 and are therefore kept out of the main method. These results support a conservative conclusion: the useful contribution is not a larger collection of gates, but an online-consistent selector with bounded local correction.

5 Discussion and Limitations

The experiments support a conservative failure-boundary interpretation. Neural feedback is not uniformly beneficial across all instances, and the neural-guided Glucose workflow alone does not

Table 5: Neural-stage full400 runtime and solver-seed robustness. Same-seed repeats use three full400 reruns with the default seed. Solver-seed robustness uses Glucose seeds 1, 2, and 3. The model, thresholds, and Local Boundary Correction guard are unchanged.

Method	Repeat	Seed	Std	Mean (s)
One-shot	48.0	48.0	0.0	47.665
Online-Consistent Selector	53.0	53.0	0.0	46.324
Old Compact	54.0	54.0	0.0	46.278
+ Local Boundary Correction	54.0	54.0	0.0	45.914

Table 6: Ablations used in the paper narrative. The official full400 boundary correction uses the 750-feature guarded rule. The no750 row is diagnostic only and has not replaced the official full400 patch.

Ablation	Scope	Result	Takeaway
No guard	boundary subset	5/26; mean 53.341 to 51.675	Local signal exists, but needs a guard
Candidate guard	full400 3-seed	opens 4 candidates; 54/200; mean 45.914	Guard keeps correction local
no750 diagnostic	trace only	opens 3 positive + 1 neutral	Diagnostic only; not official full400 patch
Local correction on/off	full400 3-seed	53.0 to 54.0 solved; mean 46.324 to 45.914	Final boundary correction claim

dominate stronger SAT solvers. Its value in the present evidence is methodological: it shows how event traces can control unsafe neural feedback and repair a narrow class of over-conservative decisions. The Online-Consistent Selector is valuable because it makes adapter activation conditional on evidence from the actual CDCL trajectory, while keeping the final intervention sparse.

Local Boundary Correction exposes a remaining limitation: conservative risk control can close useful branches. The guarded correction repairs a small number of these boundary errors, but it is not a new global selector; under the 3-seed full400 protocol it matches Old Compact solved count and reduces mean runtime. The official boundary correction uses the 750-feature guarded rule; no750 remains a diagnostic feature ablation.

The portfolio result also has a sharp claim boundary. The original single-run comparison gives a +4 portfolio advantage, but repeated CaDiCaL baselines solve 80/200 and remove the matched solved-count advantage in repeats 1 and 2. Full repeated-baseline overlap has three Local-solved CaDiCaL-unsolved instances, but only one is a Local-only gain beyond Online: `3sat_188.cnf`. March strict-60 solves all three of these instances and 184/200 overall. Other portfolio-only cases should be treated as cutoff/runtime-boundary evidence or CaDiCaL repeat variance, not neural solves. The current evidence is strongest on the full 3SAT-400 test set. Appendix results on 3SAT-300 and 3SAT-350 show that the Online-Consistent Selector does not collapse on smaller held-out sizes, and the broader appendix table includes Glucose default and Old Compact references. These remain generality and baseline robustness checks rather than a new cross-scale claim. The runtime robustness evidence is also scoped: it covers same-seed repeats and seeds 1, 2, and 3 for the neural stage plus three full portfolio and CaDiCaL repeats, not cross-machine reproducibility.

6 Conclusion

This paper studies event-conditioned neural SAT guidance as a conservative first stage for a CDCL portfolio. Rather than calling a neural model at every branching decision, the solver collects low-frequency CDCL event traces, uses an Online-Consistent Selector to decide whether event-adapted guidance should be activated, and applies a guarded Local Boundary Correction for a narrow class of over-conservative boundary cases. On held-out 3SAT-400 instances, the resulting Local5–CaDiCaL55 portfolio solves 79–80/200 instances over three repeats. Repeated standalone CaDiCaL 60s runs solve 75, 80, and 80 instances, so the portfolio evidence is boundary-sensitive rather than a robust solved-count win. A repeated March strict-60 audit solves 184/200 in every repeat and removes the current strong-baseline complementarity claim. The results suggest that solver feedback is most useful when treated as sparse evidence for risk-controlled neural intervention and failure-boundary analysis, not as an unconditional replacement for a strong SAT solver.

A Repeated Runtime Audit

Table 7 reports the scoped same-seed repeated runtime audit. This audit repeats only the fixed key instances and does not test seed sensitivity. It supports all six recovered timeouts. `3sat_196.cnf` and `3sat_46.cnf` are stable hard speedups, `3sat_188.cnf` is boundary-sensitive, and `3sat_66.cnf` remains neutral timeout evidence.

Table 7: Same-seed repeated runtime audit for the key recovered-timeout and guarded boundary-correction claims.

Instance	Comparison	Stable?	Interpretation
<code>3sat_163.cnf</code>	One-shot vs Online-Consistent Selector	yes	recovered timeout stable
<code>3sat_188.cnf</code>	One-shot vs Online-Consistent Selector	yes	recovered timeout stable
<code>3sat_189.cnf</code>	One-shot vs Online-Consistent Selector	yes	recovered timeout stable
<code>3sat_85.cnf</code>	One-shot vs Online-Consistent Selector	yes	recovered timeout stable
<code>3sat_89.cnf</code>	One-shot vs Online-Consistent Selector	yes	recovered timeout stable
<code>3sat_97.cnf</code>	One-shot vs Online-Consistent Selector	yes	recovered timeout stable
<code>3sat_188.cnf</code>	Online-Consistent Selector vs + Local Boundary Correction	mixed	boundary-sensitive
<code>3sat_196.cnf</code>	Online-Consistent Selector vs + Local Boundary Correction	yes	stable hard speedup
<code>3sat_46.cnf</code>	Online-Consistent Selector vs + Local Boundary Correction	yes	stable hard speedup
<code>3sat_66.cnf</code>	Online-Consistent Selector vs + Local Boundary Correction	yes	neutral timeout evidence

B 300/350 Generality Check

Table 8 reports the broader generalization and baseline robustness check. The 300/350 rows are single-run appendix checks under the frozen protocol. The 400 neural rows are neural-stage 3-seed solver robustness results. CaDiCaL default is included as a stronger unguided CDCL reference, not as a neural baseline. Local Boundary Correction is only evaluated on the 400-boundary setting.

C Artifact Sources

This draft is assembled from the portfolio stability audit, the portfolio claim split, the paper claim migration audit, and the existing method, experiment, related-work, and table source documents under `docs/`. The current cactus plot is a neural-stage mechanism figure, not the portfolio main result. The reproducibility package is tracked in the paper reproducibility and artifact-manifest documents. Checkpoints remain external artifacts or Git LFS candidates.

Table 8: Generalization and baseline robustness across held-out 3SAT sizes. The 300/350 rows are single-run appendix checks under the frozen protocol. The 400 neural rows are neural-stage 3-seed solver robustness results. Local Boundary Correction is only evaluated on the 400-boundary setting.

Size	Method	Solved	Mean time (s)	Median time (s)
300	Glucose default	197	20.222	18.630
300	One-shot	200	15.311	14.099
300	Online-Consistent Selector	200	7.272	6.061
300	Old Compact	200	8.907	7.597
350	Glucose default	73	47.453	60.000
350	One-shot	108	43.741	57.104
350	Online-Consistent Selector	109	33.815	40.065
350	Old Compact	103	34.918	46.533
400	Glucose default	13	57.615	60.000
400	CaDiCaL default	75	42.924	60.000
400	One-shot	48.0	47.665	60.289
400	Online-Consistent Selector	53.0	46.324	60.366
400	Old Compact	54.0	46.278	60.373
400	+ Local Boundary Correction	54.0	45.914	60.369

References

- [1] Citation Pending. Neuroback: Improving cdcl sat solving using graph neural networks. In *International Conference on Learning Representations*, 2024. Draft placeholder; replace with official BibTeX before submission.
- [2] Citation Pending. Neuroselect: Learning to select clauses in sat solvers. In *Design Automation Conference*, 2024. Draft placeholder; replace with official entry before submission.
- [3] Citation Pending. Boolean satisfiability via imitation learning, 2025. Draft placeholder; replace with official entry before submission.
- [4] Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey, 2012.
- [5] Vitaly Kurin, Saad Godil, Shimon Whiteson, and Bryan Catanzaro. Can q-learning with graph networks learn a generalizable branching heuristic for a sat solver? In *Advances in Neural Information Processing Systems*, 2020.
- [6] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *AAAI Conference on Artificial Intelligence*, 2019.
- [7] Daniel Selsam and Nikolaj Bjørner. Guiding high-performance sat solvers with unsat-core predictions, 2019.
- [8] Daniel Selsam, Matthew Lamm, Benedikt Bunz, Percy Liang, Leonardo de Moura, and David L. Dill. Learning a sat solver from single-bit supervision. In *International Conference on Learning Representations*, 2019.

- [9] Jan Tönshoff and Martin Grohe. Learning from algorithm feedback: One-shot sat solver guidance with gnns, 2025. Draft placeholder; replace with official ICLR/OpenReview entry before submission.
- [10] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.
- [11] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Satzilla: Portfolio-based algorithm selection for sat. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.
- [12] Rui Zhai and Dongdong Ge. Learning splitting heuristics in divide-and-conquer sat solvers with reinforcement learning. In *International Conference on Learning Representations*, 2025. Draft placeholder; replace with official entry before submission.